GAME PROGRAMMER

PORTFOLIO



小菅 友暉

HAL東京
ゲーム4年制学科

C++ / DirectX 11 / Unity

Profile

制作で意識していること

繰り返し作業を減らすために、
処理の役割とデータの持ち方を考える。

関心のある分野

プレイヤーの操作感と、
制作を支えるツールの仕組み。

使用経験

C++ / C#
DirectX 11
Unity / Unreal Engine

資格・受賞

基本情報技術者試験 合格
CGエンジニア検定 ベーシック
HAL東京 HEW1年次 技術力賞
paiza Bランク

Works

01 **KickLoop** 個人制作

アクションと操作感

02 **ビジュアルプログラミング** 個人制作

ノード編集・実行・保存

03 **ANNiHILATION** チーム制作

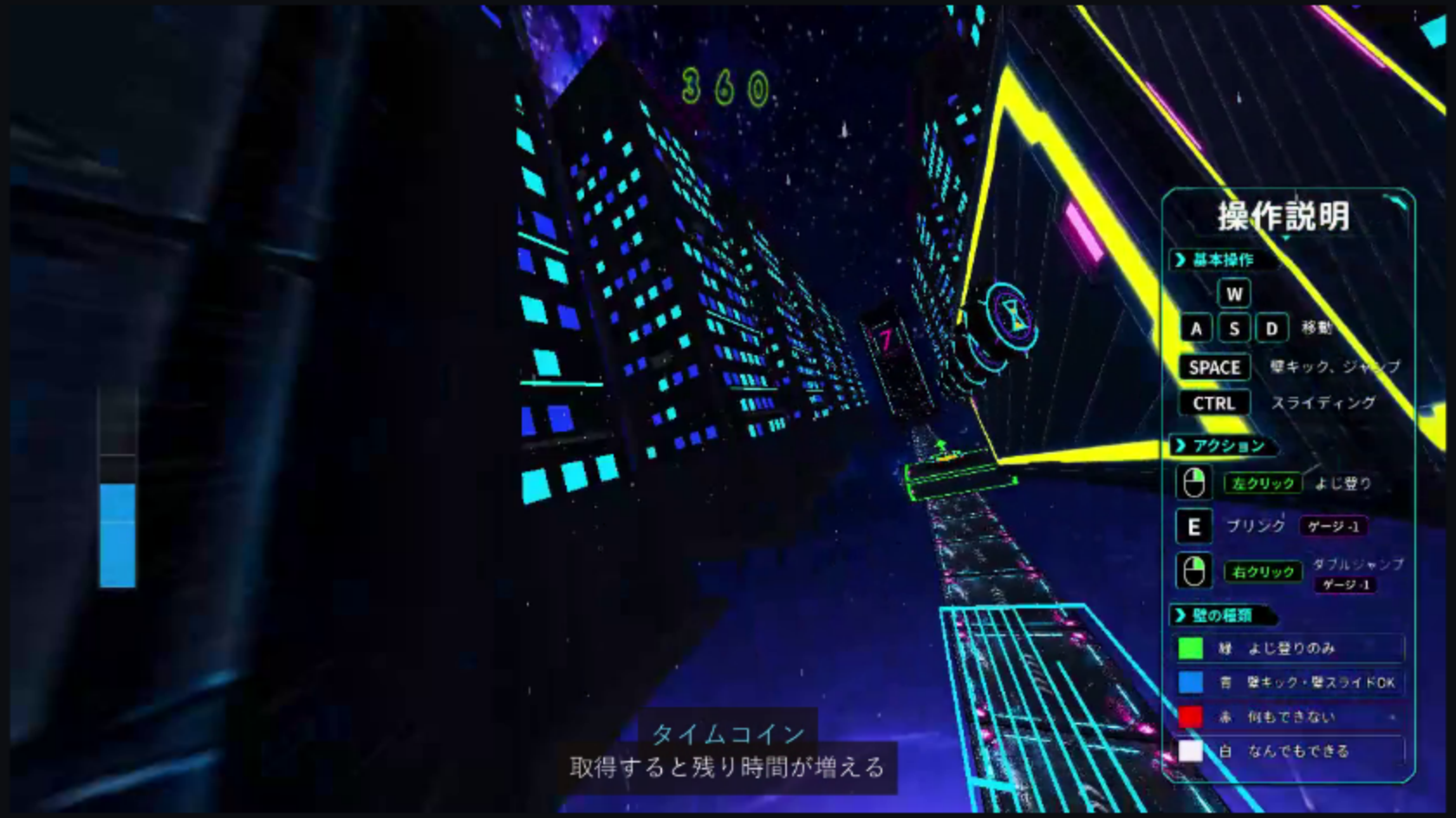
UI・画面ノイズ演出

04 **ダイナミックブレイカー** チーム制作

床配置・シャチの動作

KickLoop

個人制作 / 3か月 / C++・DirectX 11



壁キックやよじ登りをつなぎ、制限時間内のゴールを目指す一人称アクション。

アクションの処理構成

KickLoop

複数の操作が速度を変えるため、
要求を集めて、決まった順番で反映。

01

Controller

条件を調べて、
変更を要求する。

02

ActionRequest

要求を1フレーム分保持。
入力と反映を分ける。

03

Resolver

移動・重力・接地を
決まった順番で処理する。

ジャンプと壁キックの入力

KickLoop

起きた問題

Spaceを両方の処理に渡すと、
通常ジャンプの先行入力が残る。

対応

壁接触中と壁キックの猶予中は、
通常ジャンプのバッファを消す。

```
if (state.isTouchWall)
{
    m_JumpBufferFrame = 0;
    return;
}
```

先行入力 8フレーム
壁キック猶予 8フレーム
壁キック時の停止 5フレーム

通常ジャンプの入力が、壁キック後に残らないようにした。

演出とサウンド

KickLoop



GPUパーティクル

CPUから位置と数を渡す。
寿命や速度をGPUで計算。
着地やゴールの演出に使う。

CRI Atom

BGM・SE・VOICEを管理。
ゲーム中も音量を調整。

通過した障害物を再利用

KickLoop / StageBlockManager

先の区間を生成し、
通過した障害物をプールへ戻す。

回収

更新・描画を止めて待機。
次の生成で取り出して使う。

```
obstacle->SetActive(false);
obstacle->SetVisible(false);
m_ObstaclePool.push_back(obstacle);
```

再利用時に状態を初期化。プールに足りない分は新しく生成する。

粒子の位置と寿命をGPUで更新

KickLoop / gpuParticleCS.hlsl

CPU側

発生位置・数・種類を渡す。

GPU側

粒子ごとに速度・位置を更新。
寿命が尽きたら無効にする。

着地の火花や環境演出など、複数の発生パターンで使う。

```
[numthreads(256, 1, 1)]


particle.VelocityMaxLife.y
    += gravity * deltaFrame;

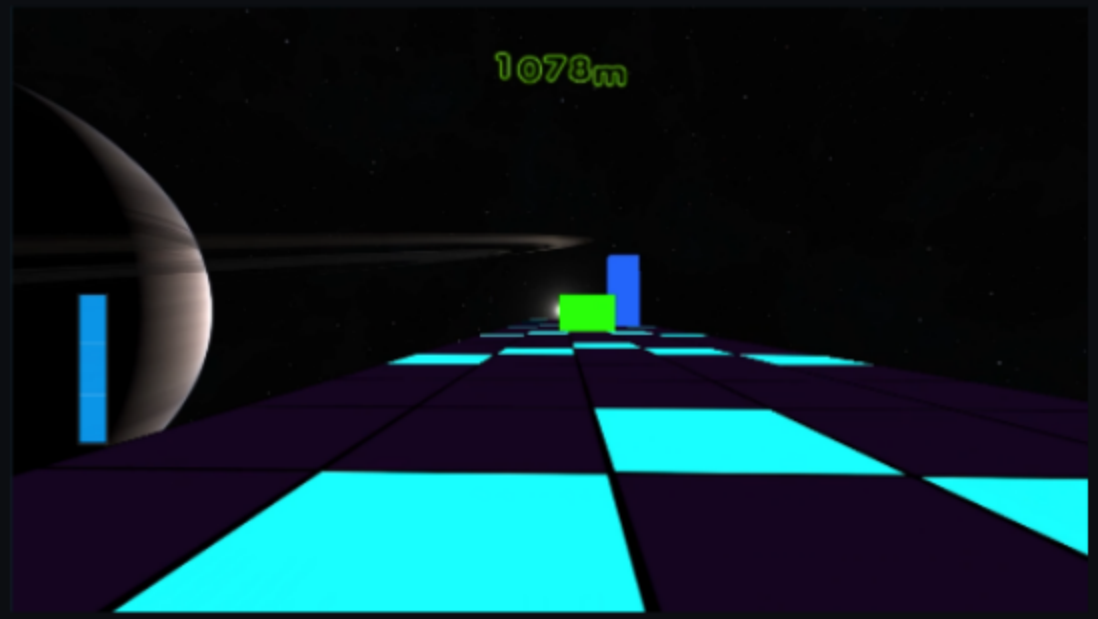
particle.PositionLife.xyz
    += particle.VelocityMaxLife.xyz
       * deltaFrame;

particle.PositionLife.w
    -= deltaFrame;
```

ステージの仕上げ

KickLoop

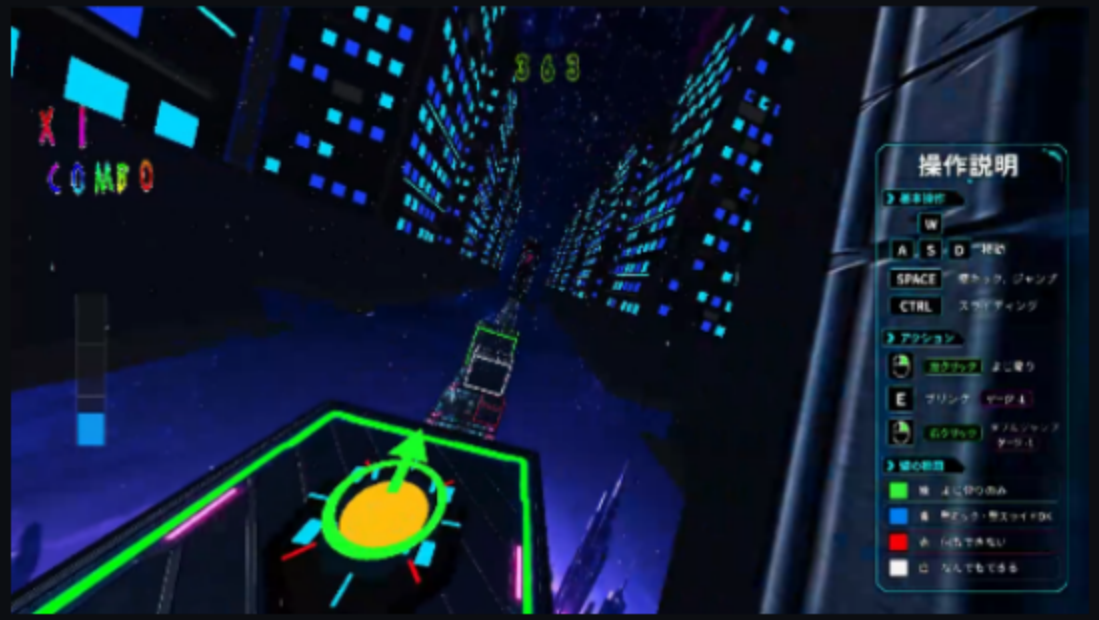
Prototype



単色の床と壁で地形を確認。

地形・障害物はJSONで管理し、調整する場所をそろえた。

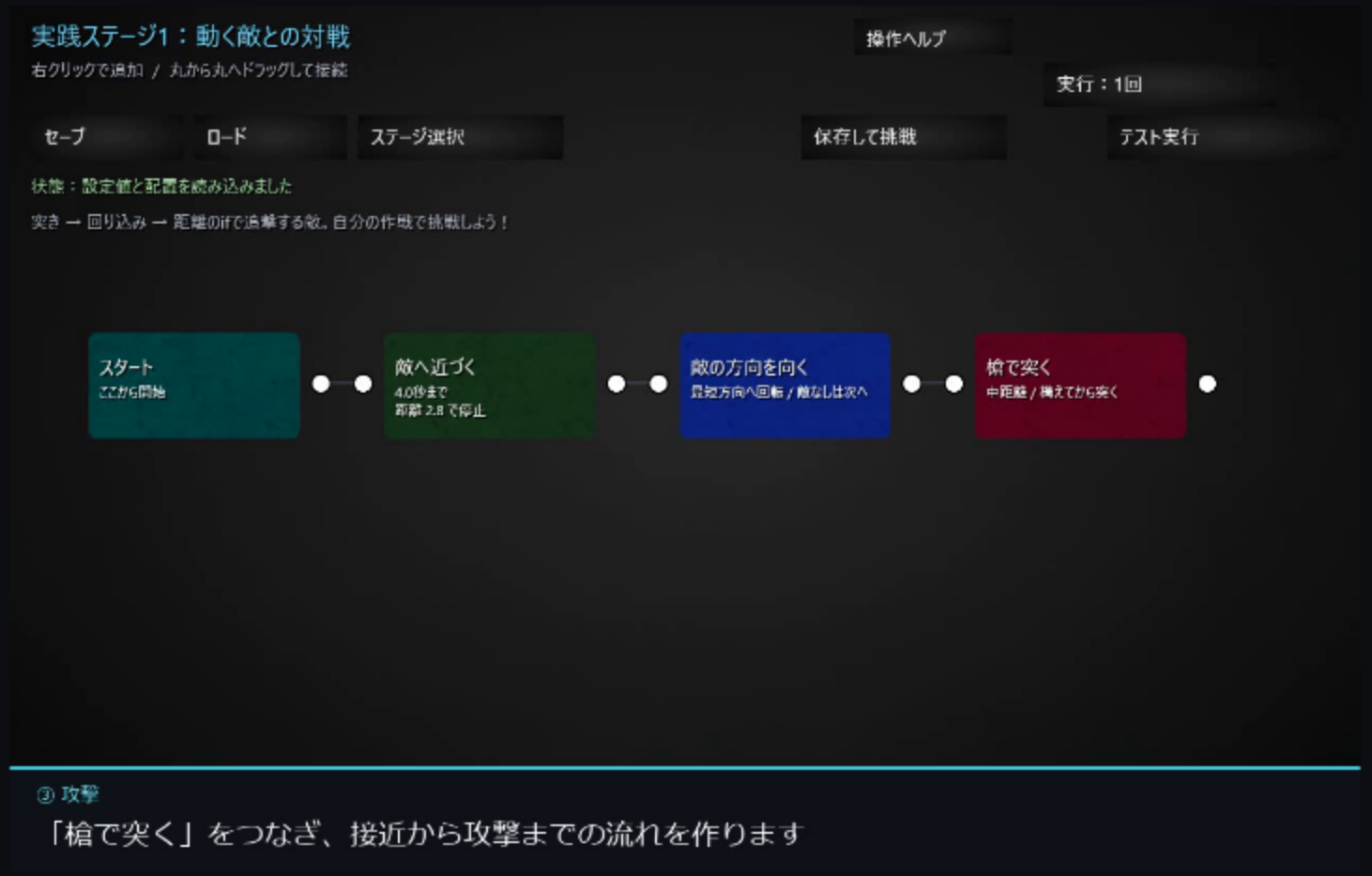
Completed



壁の色分けと操作案内を追加。

ビジュアルプログラミング

個人制作 / 半年 / C++・DirectX 11



移動・条件・攻撃のノードをつないで、キャラクターの行動を組み立てる。

編集と実行で接続の持ち方を変える

ビジュアルプログラミング

編集・保存

ノードIDで管理

配置・設定・接続をJSONへ保存。
読み込み時にIDから接続先を探す。

```
if (node && node->GetID() == id)
{
    return node;
}
```

ゲーム実行

ノードを直接参照

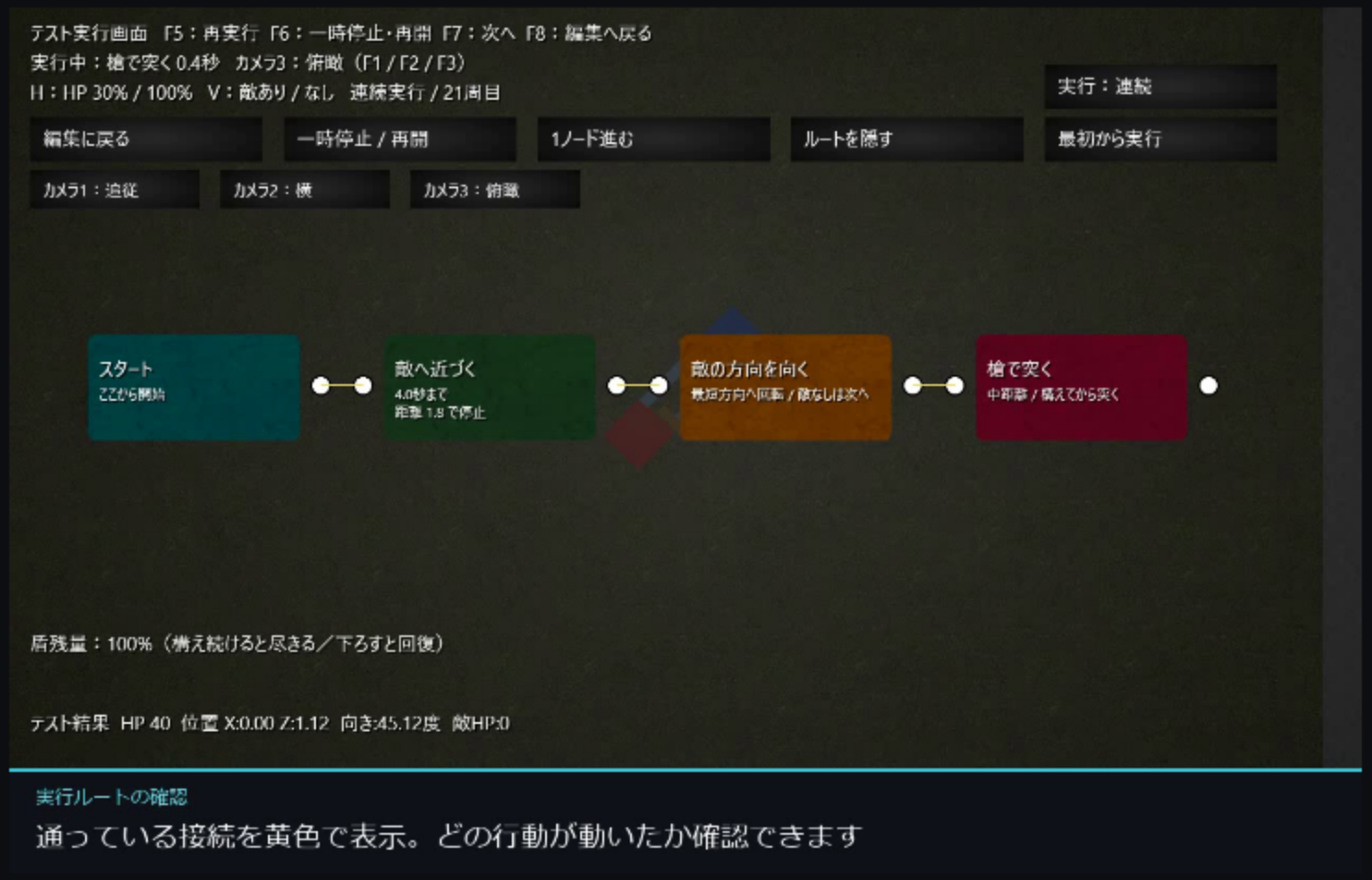
開始・条件ノードはExecute、
行動ノードはTickで進める。

```
auto *next =
    m_Current->Execute();
```

読み込み時と実行時で、接続を扱う方法を分けている。

実行経路を画面で確認

ビジュアルプログラミング



実行ルートの確認

通っている接続を黄色で表示。どの行動が動いたか確認できます

課題

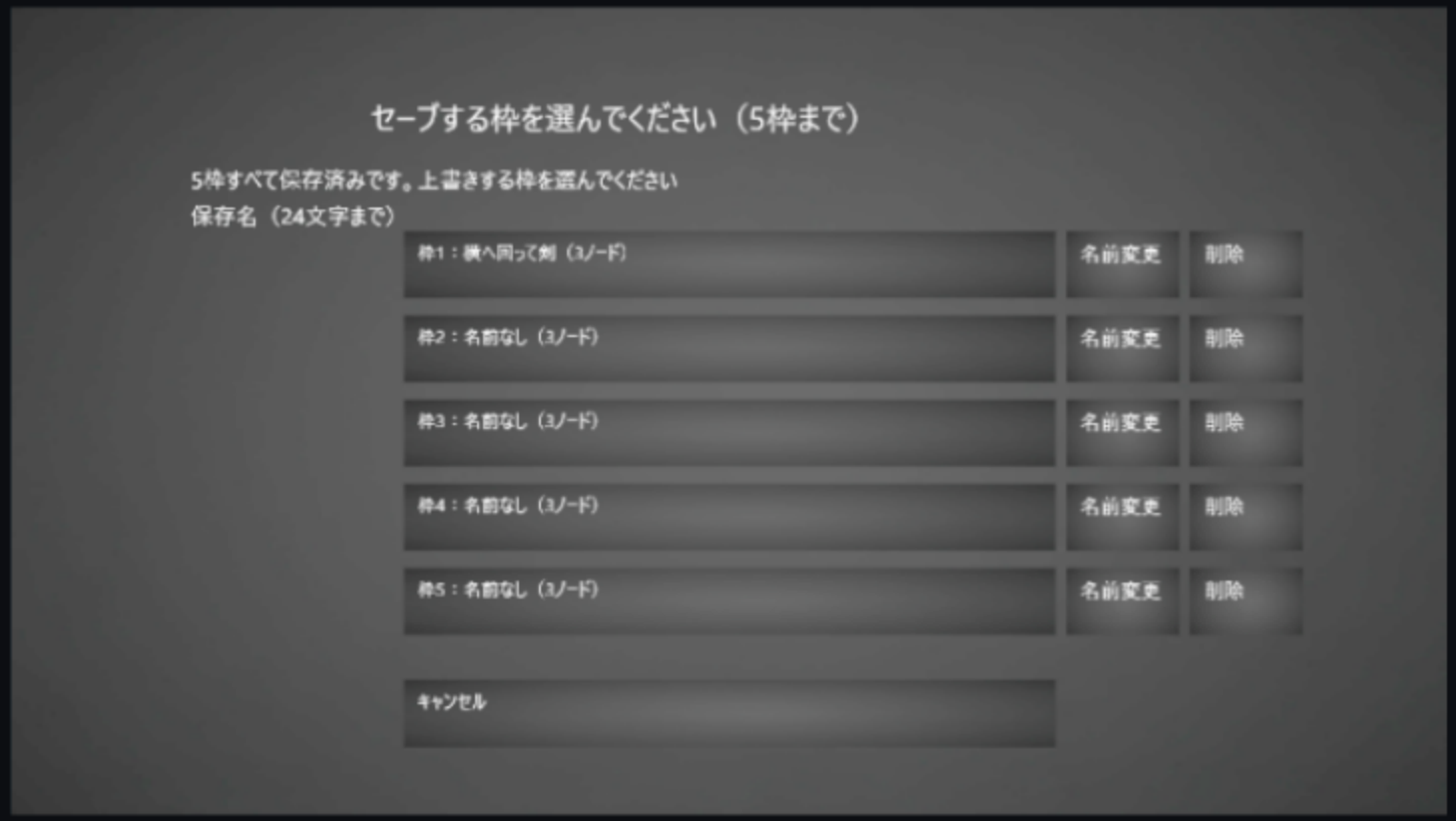
分岐の結果が見えず、
原因を追いにくかった。

対応

ノードと接続線を強調。
動きと経路を同時に確認。

作戦を名前付きで保存

ビジュアルプログラミング / NodesSaveLoad



5枠のセーブ

ノードの設定と配置を保存。
名前で作戦を見分ける。

読み込みの確認

IDや接続を検証してから、
編集中のノードを置き換える。

保存名の変更にも対応。書き込みを終えてから元のファイルを置き換える。

接続時と実行前にチェック

ビジュアルプログラミング / NodeManager・NodeRuntime

線をつなぐとき

自分自身への接続や、
循環する接続を止める。

条件ノードと通常ノードで、
使える出力ポートを分ける。

実行する前

スタート数と接続先を確認。
分岐の接続漏れも確認する。

スタートから到達する条件ノードは、
「はい」「いいえ」の両方が必要。

未使用のノードは警告。実行できない接続は、理由を画面に表示する。

時間がかかる行動を継続して実行

ビジュアルプログラミング / NodeRuntime

移動や攻撃が終わってから、
次の行動へ進む。

Begin

行動の開始時に1回。

Tick

毎フレーム進行。
実行中・完了・失敗を返す。

End

行動を終了して、
次のノードへ切り替える。

実行状態を管理し、一時停止や1ノードずつの確認に対応する。

ANNiHILATION

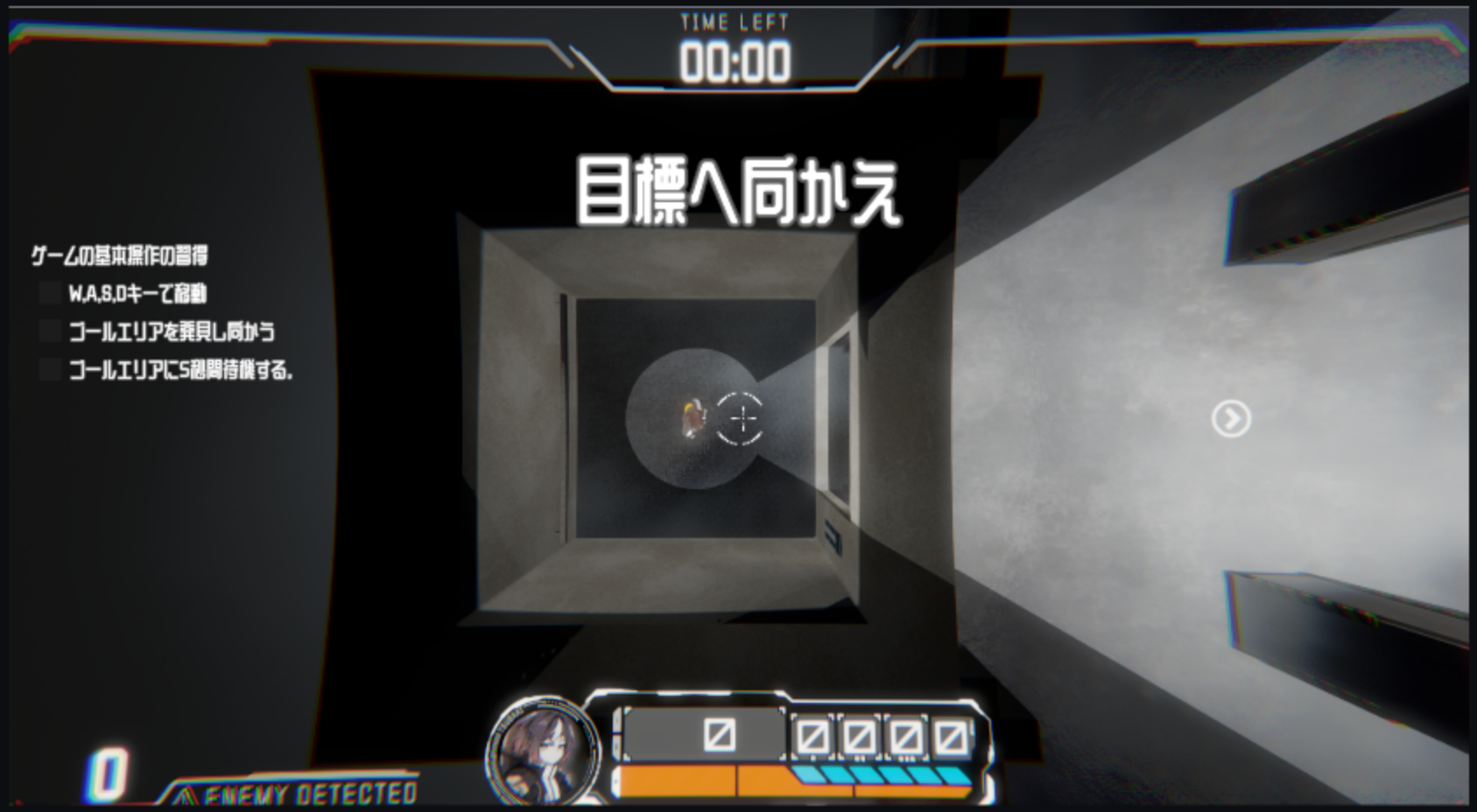
12人チーム / 約3か月 / Unity 6000.3.11f1



壁を壊したり作ったりして、敵を避けながらゴールを目指す。

担当したUI

ANNiHILATION



状態の表示

HP
アビリティゲージ
武器の情報

目標への案内

画面端のマーカーで案内。
方向と距離に合わせて
表示を変える。

UIの共通処理をまとめる

ANNiHILATION / UI_Base

共通の土台

表示・非表示と、
一定時間後に隠す処理を共有。

```
Show()
Hide()
ShowForSeconds(displayTime)
```

Canvasの表示方式に合わせて、
座標変換に使うカメラを選ぶ。

各UIの処理

HPや武器などの表示は、
個別のクラスで更新する。

ゲージを1枠ずつ埋める

ANNiHILATION / UI_AbilityGauge

回復途中の小数も、
ゲージの塗りで見せる。

各枠の塗り

現在値から枠番号を引き、
0〜1に収めて表示する。

```
float capsuleFill =
    Mathf.Clamp01(currentEnergy - i);

m_capsuleImages[i].fillAmount
    = capsuleFill;
```

例：現在値が2.5なら、2枠を満たし、次の1枠を半分まで塗る。

画面外の目標を案内する仕組み

ANNiHILATION / UI_GoalNavi

目標が見えなくても、
進む方向を画面端で伝える。

01

画面の座標へ変換

目標とプレイヤーの位置を
画面上の座標へ変換。

02

画面端との交点

目標の方向へ進めたとき、
最初にぶつかる画面端を求める。

03

表示を調整

方向に合わせて回転。
距離に合わせて大きさを変える。

画面端の余白を保ち、Canvasの座標へ変換してマーカーを配置する。

ダメージとゲームオーバーの演出

ANNiHILATION

赤いビネットの案から、
世界観に合わせたノイズへ。

チームで相談して変更。Renderer Featureを使い、ノイズの強さを時間で制御。

ダメージ時

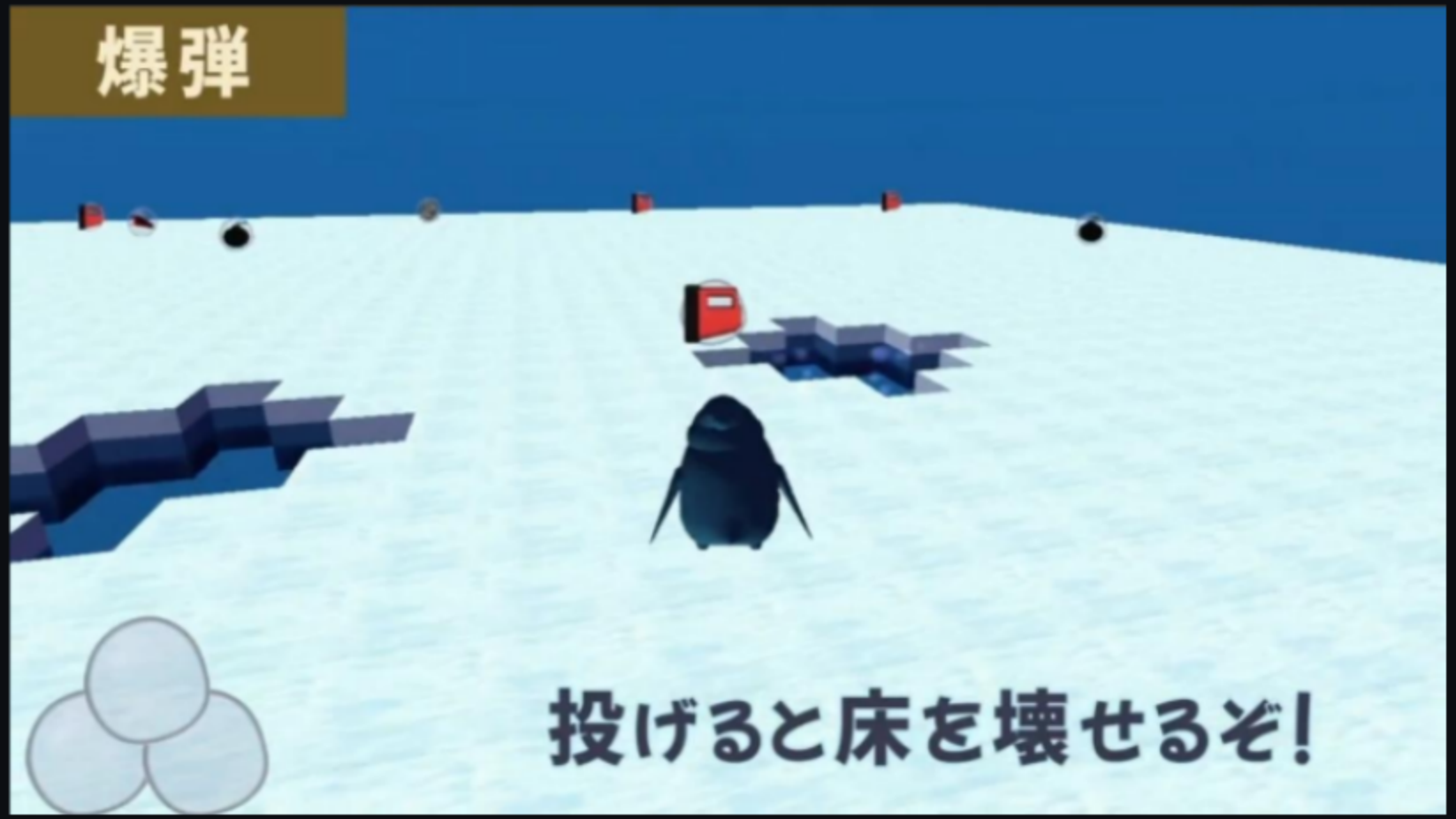
一時的にノイズを出し、
時間とともに弱める。

ゲームオーバー時

ノイズを強め、
画面の変化で状態を伝える。

ダイナミックブレイカー

チーム制作 / C++・DirectX 11



担当

field
床の配置・生成

syachi
シャチの動作制御

CRIは導入を検討・試行。
既存実装との兼ね合いで
導入には至らなかった。

その他の制作経験

タイムアタックレーシング



個人制作 / 1週間 / C++・DirectX 11
車両制御、地形生成、追従カメラ、3D音響。

スプラスター



個人制作 / 1か月 / C言語
2人対戦の状態管理とコンソールUI。

作品リンク

KickLoop	Google Drive
ビジュアルプログラミング	Google Drive
ANNiHILATION	Google Drive
ダイナミックブレイカー	Google Drive

作品名から実行ファイル・動画のフォルダへ移動できます。
素材やDLLを含めて展開してから起動してください。